Tas'heel | تسهيل
سافر الآن وأدفع لاحقاً

Assisted Checkout Merchant Integration Guide

Let your customers pay their Tasheel downpayment with a saved card, right inside your website or mobile app. No redirect.

Website | Flutter / mobile app | OTP consent | Saved cards

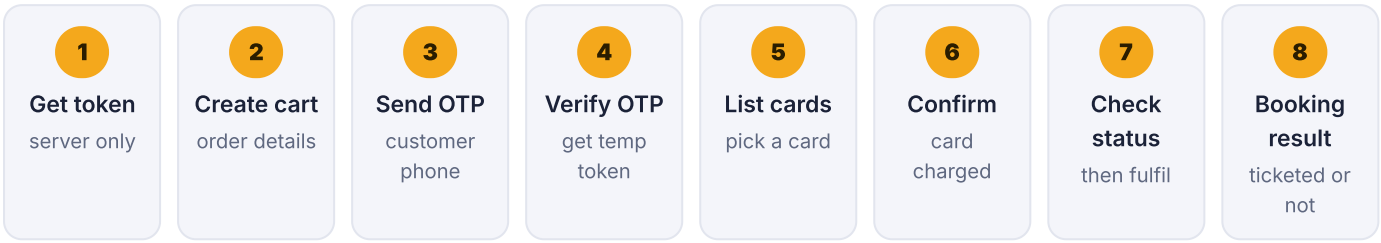
Version 2.1 · November 2026

For merchant development teams

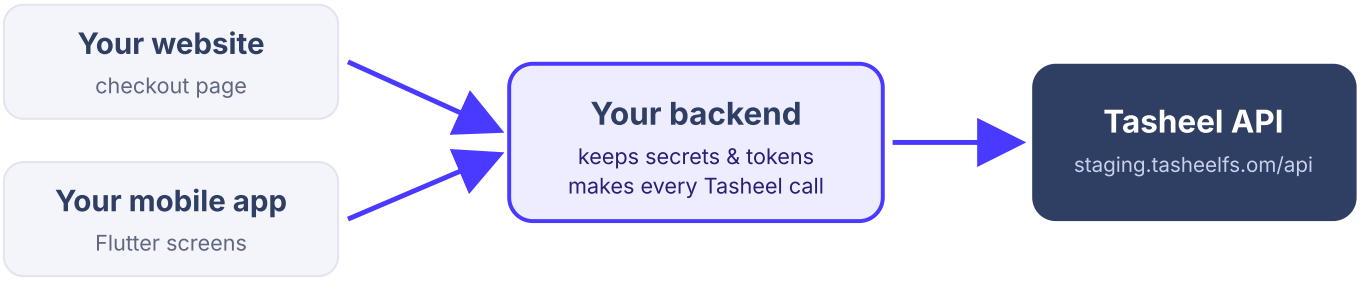
Tasheel Financial Services · Technical Team

How it works in one page

Assisted Checkout lets a customer who already has a Tasheel account pay the **downpayment** of a Buy Now, Pay Later plan without leaving your checkout. The customer proves it is them with a one-time code (OTP), picks one of the cards saved in their Tasheel account, and the card is charged immediately. Occasionally the payment needs a moment to be confirmed, or the bank asks the customer for an extra check; this guide shows how to handle both. The remaining instalments are then managed by Tasheel.



Who calls what



GOLDEN RULE

Your website and app never call Tasheel directly. They call **your backend**, and your backend calls Tasheel. The client secret, the access token and the OTP temporary token must never reach a browser or a phone.

In this guide

1. Before you start: credentials, URLs, headers
2. The 8 steps, each with request, response and what the customer sees
3. Rules and limits (OTP cooldown, token expiry, payment retries)
4. Error handling: what to show for every error code
5. Building it in your app: backend routes and Flutter example
6. Testing and go-live checklist

Before you start

What Tasheel gives you

client_id + client_secret	OAuth client used to get an access token (step 1). Keep on your server only.
merchantId	Your merchant identifier, sent when you create a cart (step 2).
merchant_uuid	Your merchant UUID, sent when you request and verify the OTP (steps 3 and 4).
Staging access	A staging environment and test customer accounts with saved cards.

Request these from the Tasheel technical team. Also tell us the **callback (webhook) URL** on your server and the server IP addresses you will call from, if we need to whitelist them.

Base URLs

Environment	Base URL
Staging (testing)	`https://staging.tasheelfs.om/api`
Production (live)	`https://tasheelfs.om/api` (confirmed by Tasheel at go-live)

All paths in this guide are relative to the base URL. Example: `POST /checkout/cart` means `POST https://staging.tasheelfs.om/api/checkout/cart`.

Two kinds of token

Token	Where it comes from	How to send it	Used for
Access token	Step 1 · `/oauth/token`	`Authorization: Bearer <access_token>`	Creating the cart. Valid until `expires_at`; cache and reuse it.
Temp token	Step 4 · OTP verify	`temp-user-access: <temp_token>`	Cards, confirm and status for **one customer**. Valid **10 minutes**.

Common headers

```
Content-Type: application/json
Accept: application/json
```

Response format

Every response has a `status` (`success` or `error`) and a machine-readable `code`. **Always build your logic on `code`**, not on the human `message` text. Errors also include an `errors` object with details.

```
{
  "status": "error",
  "code": "INVALID_OTP",
  "message": "Wrong OTP entered.",
  "errors": { "INVALID_OTP": ["Wrong OTP entered."], "remaining_attempts": [1] }
}
```

1 Get an access token

When: once, then reuse until it expires. **Customer sees:** nothing; this is server to server.

`POST` `/oauth/token` no auth

REQUEST BODY

```
{
  "client_id": 123,
  "client_secret": "your-client-secret",
  "grant_type": "client_credentials"
}
```

RESPONSE

```
{
  "access_token": "eyJ0eXAiOiJKV1Qi...",
  "token_type": "Bearer",
  "expires_at": "2026-12-01T12:00:00Z"
}
```

TIP

Store the token with its `expires_at` and request a new one a few minutes before it expires. Don't request a new token for every order.

2 Create the cart

When: the customer chooses "Pay later with Tasheel" at checkout. The cart holds the order amount and items. Keep the returned `cart_id`: you need it in step 6.

POST `/checkout/cart` — Authorization: Bearer <access_token>

REQUEST BODY

```
{
  "merchantId": "your-merchant-id",
  "customerPhone": "96891234567",
  "customerEmail": "customer@example.com",
  "quotationType": "refundable",
  "cartItems": [
    { "productId": "TRIP-MCT-IST", "quantity": 1, "price": 150, "total": 150,
      "description": "Muscat - Istanbul return + hotel" }
  ],
  "totalAmount": 150,
  "referenceCode": "your-merchant-id-ORD98765",
  "cartValidity": "02:00",
  "callback_url": "https://api.your-store.com/tasheel/webhook",
  "service_fee_pass_to_customer": true
}
```

Field	Required	Meaning
`merchantId`	Yes	Your merchant ID from Tasheel.
`customerPhone`	Yes	Customer mobile with country code, digits only (max 15). Use the same number in step 3.
`cartItems`	Yes	List of items: `productId`, `quantity` (≥1), `price`, `total`, optional `description`.
`totalAmount`	Yes	Order total in OMR.
`referenceCode`	Recommended	**Idempotency key** = your merchantId + your order number. Sending the same code again returns the existing cart instead of creating a new one.
`cartValidity`	Optional	How long the customer has to pay, `HH:mm`. Min `00:01`, max `48:00`.
`callback_url`	Recommended	Your webhook; Tasheel posts the payment result here (step 7).
`quotationType`	Optional	`refundable` or `non_refundable`.
`service_fee_pass_to_customer`	Optional	`true` = the customer pays the service fee, `false` = you absorb it.
`customerEmail`	Optional	Customer email.

RESPONSE

```
{
  "status": "success",
  "code": "CART_CREATED",
  "message": "Cart created successfully",
  "cart_id": "728ef50f-6d14-42df-83da-7b5bba574af2",   // save this → step 6
  "ref_code": "your-merchant-id-ORD98765",
  "cart_expire_at": "2026-10-25T14:10:00Z"
}
```

IGNORE REDIRECT_URL

The response may also contain a `redirect_url`. That is for the redirect flow, which you do **not** use with Assisted Checkout.

WHAT THE CUSTOMER SEES

The customer selects **Pay later with Tasheel**. Your backend creates the cart in the background, then shows the mobile number field for step 3.

VALIDATION ERRORS (422)

`VALIDATION_FAILED` with the field name in `errors`, for example an invalid `cartValidity` format.

3 Send the OTP (customer consent)

When: the customer enters their Tasheel mobile number and taps "Send code". Tasheel sends a 6-digit code by SMS.

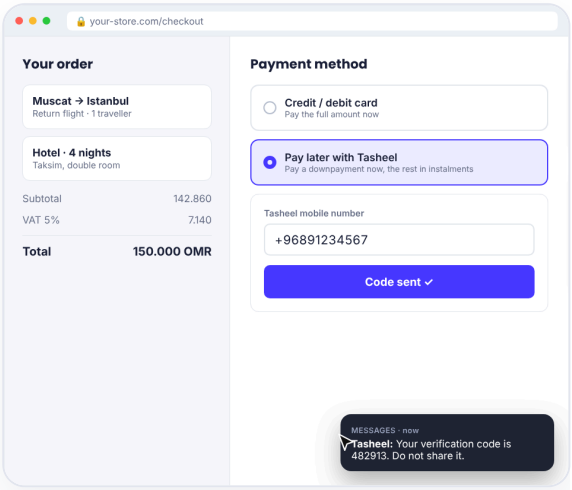
POST `/merchant/temp-auth/request` — no auth

REQUEST BODY

```
{
  "merchant_uuid": "84b5edc9-766d-....-dadcc5a7c6c7",
  "mobile": "96891234567"
}
```

RESPONSE

```
{
  "status": "success",
  "code": "OTP_SENT",
  "message": "OTP sent successfully",
  "otp_id": 98          // save → step 4
}
```



Customer enters their number and receives the code by SMS

COOLDOWN: ONE CODE EVERY 2 MINUTES

If you call this again within 2 minutes, no new SMS is sent. You get `OTP_ALREADY_SENT` with the **same** `otp_id` and the seconds to wait in `message`. Reuse that `otp_id`; the earlier code is still valid until `expires_at`. Show a "Resend code in 1:41" timer in your UI.

```
{
  "status": "success",
  "code": "OTP_ALREADY_SENT",
  "message": "OTP already sent. Please wait 101 seconds before requesting again.",
  "otp_id": 98,
  "expires_at": "2026-10-25T06:16:14.000000Z"
}
```

HTTP	code	Show the customer
403	`HAS_OVERDUE`	"You have an overdue Tasheel instalment. Please settle it in the Tasheel app or choose another payment method."
404	`NOT_FOUND`	"No Tasheel account found for this number." Offer to register in the Tasheel app.
400	`VALIDATION_FAILED`	Highlight the invalid field (e.g. mobile missing).
500	`OTP_CREATION_FAILED`	"We couldn't send the code. Please try again."

4 Verify the OTP

When: the customer types the 6-digit code. On success you receive the **temp token** that unlocks steps 5 to 7 for this customer.

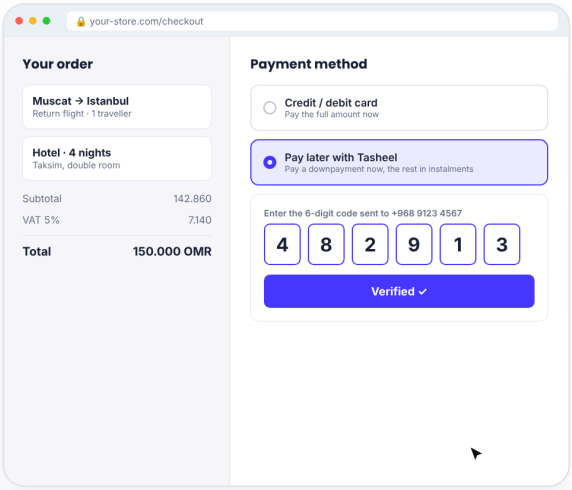
`POST` `/merchant/temp-auth/verify` — no auth

REQUEST BODY

```
{
  "mobile": "96891234567",
  "otp": "482913",
  "merchant_uuid": "84b5edc9-766d-...-dadcc5a7c6c7",
  "otp_id": 98
}
```

RESPONSE

```
{
  "status": "success",
  "code": "OTP_VERIFIED",
  "temp_token": "80428f...1b5ea5",
  "expires_at": "2026-10-25T06:39:49Z"
}
```



Customer enters the 6-digit code

TEMP_TOKEN: VALID 10 MINUTES, ONE CUSTOMER

Keep it in your backend session for this order. Send it as `temp-user-access: <temp_token>` in steps 5, 6 and 7. When it expires, start again from step 3.

MAXIMUM 2 WRONG ATTEMPTS

A wrong code returns `INVALID_OTP` with `remaining_attempts`. After 2 wrong attempts the OTP session is closed (`TOO_MANY_WRONG_ATTEMPTS`) and you must request a new code (step 3).

HTTP	code	What to do
`400`	`INVALID_OTP`	"Wrong code. 1 attempt left." Clear the boxes.
`429`	`TOO_MANY_WRONG_ATTEMPTS`	"Too many attempts." Show a "Send new code" button (step 3).
`400`	`OTP_EXPIRED`	"Code expired." Request a new code.
`400`	`SESSION_ALREADY_USED`	This code was already used. Request a new code.
`404`	`INVALID_SESSION`	Wrong `otp_id` for this mobile. Request a new code.
`403`	`MERCHANT_MISMATCH`	The OTP was requested with a different `merchant_uuid`. Fix your configuration.
`400 / 422`	`VALIDATION_FAILED`	Missing field, invalid `merchant_uuid`, or code not 6 digits.
`404`	`NOT_FOUND`	No user for this mobile.

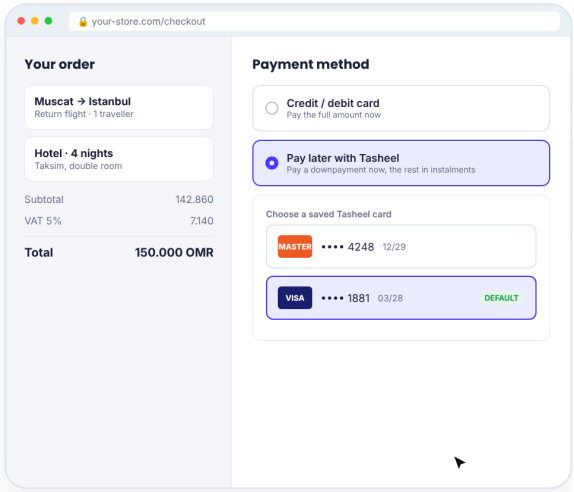
5 Show the customer's saved cards

When: right after OTP verification. Only masked card data is returned (brand, last 4 digits, expiry).

`GET` `/assisted/cards` — temp-user-access: <temp_token>

RESPONSE

```
{
  "status": "success",
  "code": "CARDS_RETRIEVED",
  "data": [
    { "id": 3, "token": "tok_125",
      "last4": "4248", "brand": "Master",
      "exp_month": 12, "exp_year": 2029,
      "is_default": true },
    { "id": 7, "token": "tok_311",
      "last4": "1881", "brand": "Visa",
      "exp_month": 3, "exp_year": 2028,
      "is_default": false }
  ]
}
```



Pre-select the card with `is_default: true`

Display each card as "**Visa •••• 1881 · 03/28**" and pre-select the default one. Use the card's `id` in step 6.

NO CARDS?

If `data` is empty, ask the customer to add a card in the Tasheel app, then tap "Refresh". Confirm returns `NOT_FOUND` "No card found, please add card to your Tasheel account" in this case.

Optional: make the selected card the default

`POST` `/assisted/cards/{id}/make-default` — temp-user-access: <temp_token>

```
{ "status": "success", "code": "DEFAULT_CARD_UPDATED", "message": "Default card updated",
"default_card_id": "7" }
```

HTTP	code	What to do
401	`TOKEN_MISSING`	Your backend didn't send the `temp-user-access` header. Fix the request.
401	`INVALID_TOKEN`	Temp token expired (10 min). Ask the customer to verify again (step 3).
404	`NOT_FOUND`	Card not found / not owned by this customer. Reload the card list.

6 Confirm and charge the downpayment

When: the customer taps "Pay". The selected saved card is normally **charged immediately**, with no extra screen for the customer. Sometimes the result is not final straight away; see **"When the payment is not final straight away"** two pages on.

`POST` `/assisted/checkout/confirm` temp-user-access: <temp_token>

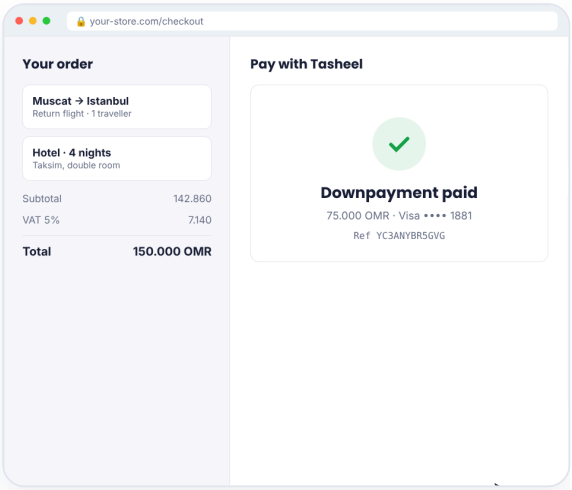
REQUEST BODY

```
{
  "cart_uuid": "728ef50f-6d14-42df-83da-7b5bba574af2",
  "default_card_id": 7
}
```

`cart_uuid` = the `cart_id` from step 2.
`default_card_id` = the `id` of the card the customer picked.

RESPONSE

```
{
  "status": "success",
  "code": "PAYMENT_SUCCESS",
  "message": "Payment successful",
  "data": {
    "payment_track": "YC3ANYBR5GVG",
    "amount": 75,
    "currency": "OMR",
    "quotation_id": 118,
    "cart_uuid": "728ef50f-...",
    "transaction_id": "PMB-987654321"
  }
}
```



Show the result with the amount and reference

PREVENT DOUBLE PAYMENT

Disable the Pay button after the first tap and wait for the response (allow up to 60 seconds). Store `payment_track` against your order straight away. If the call times out on your side, **do not call Confirm again**: check the status first.

IF THE PAYMENT FAILS: RETRY RULES

Attempts are counted per **customer + cart + card**. After a failure, wait **1 minute** (`retry_after_seconds` tells you exactly how long), then **one retry** is allowed with the same card. After that, ask the customer to pick a different card.

Confirm errors

HTTP	code	What to do
`429`	`PAYMENT_FAILED` (with `retry_after_seconds`)	"Payment failed. You can try again in 37 seconds." Show a countdown, then allow one retry.
`429`	`TOO_MANY_ATTEMPTS`	Retry already used with this card. Ask the customer to choose another card.
`402`	`PAYMENT_FAILED`	Card declined. Show the message, wait `retry_after_seconds`.
`402`	`PAYMENT_FAILED` + `provider_error`	Declined by the backup payment gateway. Show the message; the customer can try another card.
`409`	`PAYMENT_IN_PROGRESS`	An earlier payment for this cart is still being verified. Do not charge again; check status with `txnId` = `data.payment_track`.
`500`	`PAYMENT_FAILED`	Unexpected processing error. Check status (step 7) first; if it is not `pending` or `completed`, wait 1 minute before retrying with the same card.
`503`	`SECONDARY_GATEWAY_UNAVAILABLE`	Both payment gateways are down; no money was taken. Ask the customer to try again later.
`400`	`NOT_FOUND` (card)	Card not found, or no saved cards. Reload cards / ask the customer to add a card in the Tasheel app.
`400`	`EXPIRED_CART`	Cart validity ended. Create a new cart (step 2) and continue.
`403`	`INVALID_CART`	Cart belongs to another customer. The phone in step 2 and step 3 must match.
`404`	`INVALID_CART_REF`	Unknown `cart_uuid`. Check you send the `cart_id` from step 2.
`400`	`INVALID_AMOUNT`	Payment amount is invalid. Contact Tasheel support with the cart ID.
`400 / 500`	`PAYMENT_GATEWAY_NOT_FOUND` / `CONFIGURATION_ERROR`	Tasheel-side issue. Show a friendly error and contact Tasheel support.
`401`	`TOKEN_MISSING` / `INVALID_TOKEN`	Temp token missing or expired. Re-verify the customer (step 3).
`404`	`NOT_FOUND` (user)	User not found. Restart from step 3.
`422`	`VALIDATION_ERROR`	`cart_uuid` or `default_card_id` missing.

When the payment is not final straight away

Most payments return `PAYMENT_SUCCESS`. Your integration must also handle these responses from Confirm. **None of them is a failure**, and in some of them the customer's card has already been charged.

HTTP	code	Meaning	What to do
`202`	`PAYMENT_PENDING`	Card accepted; the bank has not confirmed yet (slow bank or timeout). Money may already be taken.	Show "Verifying your payment". Poll Get Status every 10–15 s. Never call Confirm again or cancel the order.
`202`	`PAYMENT_ACTION_REQUIRED`	The bank wants the customer to complete an OTP / 3-D Secure check.	Open `data.checkout_url`. The customer returns to your `cart_return_url`; then check status.
`200`	`PAYMENT_FALLBACK_INITIALIZED`	Main gateway unreachable; Tasheel opened a secure page on the backup gateway.	Open `data.checkout_url`. The customer enters card details there; then check status.
`409`	`PAYMENT_IN_PROGRESS`	An earlier payment for this cart is still being verified.	Do not charge again. Check status with `txnId` = `data.payment_track`.

PENDING RESPONSE

```
{
  "status": "pending",
  "code": "PAYMENT_PENDING",
  "message": "Payment status is being verified...",
  "data": {
    "payment_track": "YC3ANYBR5GVG",
    "amount": 75, "currency": "OMR",
    "transaction_id": null,
    "retry_after_seconds": 10
  }
}
```

HOW TO WAIT FOR THE RESULT

1. Save `payment_track`.
2. Call Get Status with `txnId` = `payment_track` every 10–15 seconds. Each call checks the payment gateway, so most payments settle within about **2 minutes**.
3. Still `pending` after **5 minutes**? Stop polling, keep the order on hold and tell the customer the result will follow. The webhook to your `callback_url` brings the final result.
4. `completed` → fulfil. `cancel` → let the customer pay again.

NEVER CHARGE TWICE

While a payment is pending, do not call Confirm again for the same cart (you get `409 PAYMENT_IN_PROGRESS`) and do not cancel the booking. A pending payment starts no cooldown and uses no retry.

RETURN URL AFTER A GATEWAY PAGE

The customer comes back to your `cart_return_url` with `?txnId=<payment_track>&status=true|false`. `status=false` can also mean the payment was still being verified, so always confirm with Get Status.

7 Check the status, then fulfil the order

Never issue the ticket or ship the order based only on what the app shows. **Confirm on your server** first. Tasheel also notifies your `callback_url`.

WEBHOOK TASHEEL SENDS TO YOUR CALLBACK_URL

```
POST https://api.your-store.com/tasheel/webhook
{
  "txnId": "YC3ANYBR5GVG",
  "status": true
}
```

Reply `200` quickly, then call Get Status to confirm. `status: false` means the payment failed. No webhook is sent while a payment is still pending.

ORDER CONFIRMED

Once status is `completed`, show "Order confirmed" and issue the ticket or booking. Then report the result to Tasheel (step 8).

GET `/checkout/get-status` — temp-user-access: <temp_token>

REQUEST BODY (SEND AT LEAST ONE)

```
{
  "cartId": "728ef50f-6d14-42df-83da-7b5bba574af2",   // cart_id from step 2
  "refCode": "your-merchant-id-ORD98765",             // your referenceCode
  "txnId": "YC3ANYBR5GVG"                            // from the webhook
}
```

RESPONSE (SHORTENED)

```
{
  "message": "Cart status",
  "data": {
    "uuid": "728ef50f-6d14-42df-83da-7b5bba574af2",
    "referenceCode": "your-merchant-id-ORD98765",
    "payment": {
      "txn_id": "YC3ANYBR5GVG", "status": "completed",
      "amount": 75.00, "final_amount": 75.00, "currency": "OMR", "try": 1
    },
    "quotation": { "total_amount": 150.00, "plan": { "installments": [ ... ] }, ... },
    "user": { ... }
  }
}
```

payment.status	Meaning	Your action
`completed`	Downpayment paid	Fulfil the order (issue ticket, confirm booking). The remaining instalments are collected by Tasheel.
`pending`	Payment started, not final; the card may already be charged	Keep the order on hold. Check again every 10–15 s for up to 5 minutes, then wait for the webhook. Do not fulfil, cancel or charge again.
`cancel`	Rejected or declined	Do not fulfil. Let the customer try again or pay another way.
`unknown`	Treat as cancel	Do not fulfil.

8 Report the booking result

When: after the downpayment is `completed` and you have tried to issue the ticket or booking. Tell Tasheel whether it was issued. If it was **not**, Tasheel refunds the customer's downpayment automatically, so the customer never pays for a booking they did not get.

POST `/checkout/booking-status` — Authorization: Bearer <access_token>

Use the server access token from step 1 (not the temp token). The cart must belong to your merchant account.

REQUEST BODY

```
{
  "cartId": "728ef50f-6d14-42df-83da-7b5bba574af2",
  "status": "TICKETED",  // or "NOT_TICKETED"
  "bookingReference": "PNR123456",
  "timestamp": "2026-11-03T10:30:00Z"
}
```

Field	Rule
`cartId` / `refCode`	Send one (`cartId` preferred). If you send both, they must point to the same cart.
`status`	Required. Exactly `TICKETED` or `NOT_TICKETED`.
`bookingReference`	Optional. Your booking reference or PNR (max 255).
`reason`	Optional. Why the booking failed (max 2000).
`timestamp`	Optional. When the result happened, ISO-8601, UTC preferred.

RESPONSE (NOT_TICKETED)

```
{
  "status": "success",
  "code": "BOOKING_STATUS_RECORDED",
  "message": "Booking failure
recorded...",
  "data": {
    "cartId": "728ef50f-...",
    "bookingStatus": "NOT_TICKETED",
    "refund": {
      "refundRequestId": 172,
      "amount": "13.000",
      "currency": "OMR",
      "status": "completed"
    },
    "duplicate": false
  }
}
```

`refund.status` is `completed` or `pending`. For `TICKETED` the response is the same with `"refund": null`.

status	What Tasheel does
`TICKETED`	Records the booking. No refund. The customer's plan continues as normal.
`NOT_TICKETED`	Records the failure and refunds the full downpayment, **including the service fee**, to the customer's card. `refund.status` is `completed` once the gateway confirms, otherwise `pending` (Tasheel retries automatically).

SEND IT ONLY FOR A COMPLETED PAYMENT

Call this endpoint only after Get Status says `completed`. For a payment that is still `pending` you get `409 DOWN_PAYMENT_NOT_SUCCESSFUL`: wait for the final result first.

Booking status errors

FINAL AND SAFE TO RETRY

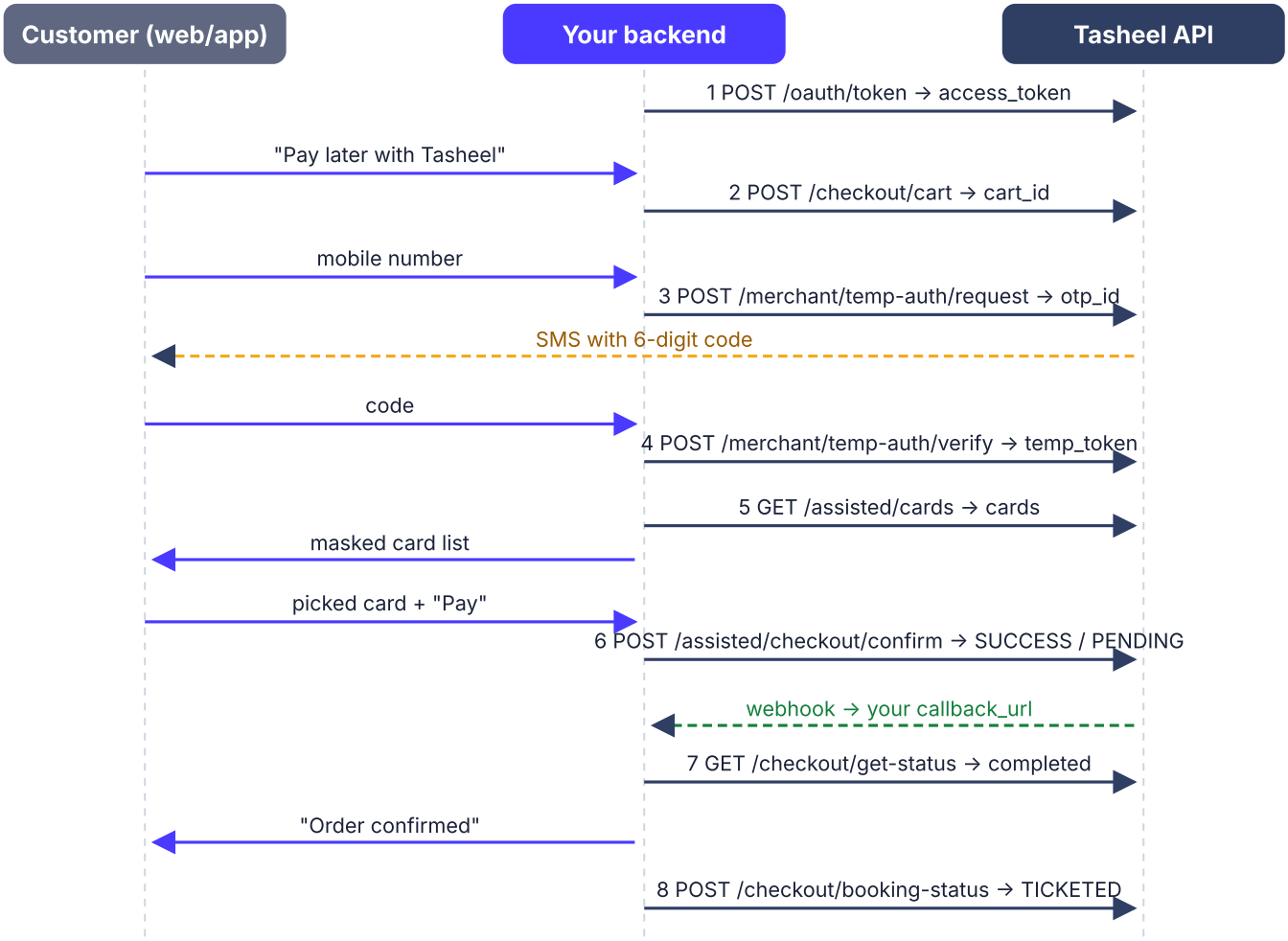
The first status you send is final: sending the opposite one later returns `409 BOOKING_STATUS_CONFLICT`. Sending the same request again is safe; it returns `BOOKING_STATUS_ALREADY_PROCESSED` with `"duplicate": true` and never creates a second refund.

HTTP	code	What to do
`200`	`BOOKING_STATUS_ALREADY_PROCESSED`	Same request sent before (`duplicate: true`). Treat as success.
`401`	`UNAUTHENTICATED`	Access token missing or expired. Get a new one (step 1) and retry.
`403`	`UNAUTHORIZED`	The cart belongs to another merchant. Check the `cartId` .
`404`	`CART_NOT_FOUND`	No cart for this `cartId` / `refCode` , or it was deleted.
`409`	`DOWN_PAYMENT_NOT_SUCCESSFUL`	The downpayment is not `completed` (failed or still pending). Check status; send the booking result only after it is completed.
`409`	`BOOKING_STATUS_CONFLICT`	The opposite status was already recorded and cannot be reversed. Contact Tasheel support if it was a mistake.
`422`	`INVALID_BOOKING_STATUS` / `VALIDATION_FAILED`	`status` must be exactly `TICKETED` or `NOT_TICKETED` ; send `cartId` or `refCode` ; check field lengths and the timestamp format.
`422`	`CART_IDENTIFIER_MISMATCH`	`cartId` and `refCode` point to different carts. Send only `cartId` .
`422`	`REFUND_INITIATION_FAILED`	Only for `NOT_TICKETED` : Tasheel could not find a refundable card payment for the cart. Contact Tasheel support with the `cartId` .
`429`	Rate limit	Too many requests (25 per minute). Retry with backoff.
`500`	`BOOKING_STATUS_PROCESSING_FAILED`	Unexpected Tasheel error. Retry the same request; it is safe to repeat.

Rules and limits at a glance

Rule	Limit	What your app should do
OTP resend	1 code per 2 minutes	Show a resend timer. On `OTP_ALREADY_SENT`, reuse the returned `otp_id`.
Wrong OTP	2 wrong attempts per code	Show remaining attempts; after the limit, request a new code.
Temp token	10 minutes	Keep the card-pick and pay steps quick. On `INVALID_TOKEN`, verify again.
Payment retry	1-minute cooldown, 1 retry per customer + cart + card	Countdown from `retry_after_seconds`; then suggest another card.
Pending payment	Blocks a new Confirm on the same cart for up to 30 minutes	Poll Get Status every 10–15 s; on `PAYMENT_IN_PROGRESS`, show the pending screen.
Cart validity	`cartValidity`: 1 minute to 48 hours	On `EXPIRED_CART`, create a new cart.
Idempotency	One cart per `referenceCode`	Use merchantId + your order number; safe to retry Create Cart.
Overdue customers	Blocked	On `HAS_OVERDUE`, offer another payment method.
Booking result	Once per cart; final	Send `TICKETED` or `NOT_TICKETED` only after `completed`. `NOT_TICKETED` refunds the downpayment.
Rate limits	`/oauth/token` 30/min	Cache the access token.

The full flow in one picture



Building it in your website and app

Expose a few simple routes on **your** backend. Your website and your Flutter app use the same routes, so the Tasheel logic is written once.

Your backend route (example)	Calls Tasheel	Returns to the app
`POST /tasheel/start` {orderId, mobile}	Token (cached) → Create Cart → Request OTP	`otp_id`, resend wait time
`POST /tasheel/verify` {orderId, otp}	Verify OTP → List cards	Masked card list (keep temp_token on server)
`POST /tasheel/default-card` {cardId}	Make default (optional)	OK
`POST /tasheel/pay` {orderId, cardId}	Confirm → Get Status	`completed` / `pending` (+ `payment_track`) / `checkout_url` / error + retry seconds
`GET /tasheel/status` {orderId}	Get Status (`txnId` = saved `payment_track`)	`completed` / `pending` / `cancel`
`POST /tasheel/booking-result` {orderId, ticketed, pnr}	Booking status (server access token)	Recorded; refund details if not ticketed
`POST /tasheel/webhook`	(called by Tasheel) → Get Status	200 OK; updates the order

Flutter example (app → your backend)

```
// The app only talks to YOUR backend. No Tasheel secrets or tokens in the app.
final api = MerchantApi('https://api.your-store.com');

// Screen 1 — phone number
final start = await api.post('/tasheel/start', {'orderId': orderId, 'mobile': mobile});
// start['code'] == 'HAS_OVERDUE' → show "settle overdue instalment" message

// Screen 2 — 6-digit code
final verify = await api.post('/tasheel/verify', {'orderId': orderId, 'otp': code});
final cards = verify['cards'];  // [{id, brand, last4, exp_month, exp_year, is_default}]

// Screen 3 — choose card, tap Pay (disable button while waiting)
final res = await api.post('/tasheel/pay', {'orderId': orderId, 'cardId': selected.id});

// Screen 4 — result
if (res['status'] == 'completed') showSuccess(res);
else if (res['status'] == 'pending') pollUntilFinal(orderId); // "Verifying your payment", GET
/tasheel/status every 10 s, max 5 min
else if (res['checkoutUrl'] != null) openSecurePage(res['checkoutUrl']); // then poll status
else if (res['retryAfterSeconds'] != null) showRetryCountdown(res['retryAfterSeconds']);
else showError(res['code']);
```

Backend example (pseudo-code for /tasheel/pay)

```
session  = loadTasheelSession(orderId)          // cart_id, temp_token saved in steps 2 & 4
confirm  = POST {BASE}/assisted/checkout/confirm
             headers: temp-user-access = session.temp_token
             body:    { cart_uuid: session.cart_id, default_card_id: cardId }
if confirm.data?.payment_track: saveOrderPayment(orderId, confirm.data.payment_track)
if confirm.code in ["PAYMENT_SUCCESS", "PAYMENT_PENDING", "PAYMENT_IN_PROGRESS"]:
    status = GET {BASE}/checkout/get-status  body: { txnId: confirm.data.payment_track }
    if status.data.payment.status == "completed": fulfilOrder(orderId)
    return status.data.payment.status        // "pending" → the app polls /tasheel/status
if confirm.code in ["PAYMENT_ACTION_REQUIRED", "PAYMENT_FALLBACK_INITIALIZED"]:
    return { checkoutUrl: confirm.data.checkout_url }
else:
    return { code: confirm.code, retryAfterSeconds: confirm.errors.retry_after_seconds?[0] }
```

Testing and go-live checklist

Test on staging

- [ ] Full happy path: token → cart → OTP → verify → cards → confirm → webhook → status `completed`.
- [ ] Create Cart twice with the same `referenceCode` returns the same cart.
- [ ] Request OTP twice within 2 minutes → `OTP_ALREADY_SENT`, same `otp_id`, resend timer works.
- [ ] Enter a wrong code twice → `INVALID_OTP`, then `TOO_MANY_WRONG_ATTEMPTS`; "Send new code" works.
- [ ] Wait more than 10 minutes after verify → `INVALID_TOKEN`; app restarts from the phone screen.
- [ ] Customer with no saved cards → helpful message.
- [ ] Customer with overdue instalment → `HAS_OVERDUE` message.
- [ ] Expired cart (short `cartValidity`) → `EXPIRED_CART`, new cart is created.
- [ ] Failed payment → countdown from `retry_after_seconds`, one retry, then "choose another card".
- [ ] Pay button cannot be tapped twice.
- [ ] Pending payment (ask Tasheel to simulate a slow charge): "Verifying your payment" screen, status polling, order fulfilled when `completed`; a second Pay returns `PAYMENT_IN_PROGRESS`.
- [ ] Card that needs a bank OTP: `checkout_url` opens, customer returns to your `cart_return_url`, status confirmed.
- [ ] Same flow works on the website and in the Flutter app (iOS and Android).
- [ ] Booking result: `TICKETED` is recorded (a repeat returns `BOOKING_STATUS_ALREADY_PROCESSED`); `NOT_TICKETED` on a test order refunds the downpayment; a pending payment returns `DOWN_PAYMENT_NOT_SUCCESSFUL`.

Before going live

- [ ] Production credentials received; production base URL confirmed with Tasheel.
- [ ] `client_secret`, access token and `temp_token` exist only on your server; not in app builds, JavaScript or logs.
- [ ] All Tasheel calls use HTTPS with a 30-second timeout.
- [ ] Webhook URL is public, HTTPS, returns 200 quickly and always confirms with Get Status.
- [ ] Orders are fulfilled only when `payment.status` is `completed`.
- [ ] Every paid order sends its booking result (`TICKETED` or `NOT_TICKETED`), retried automatically on 5xx or timeout.
- [ ] You store `cart_id`, `payment_track` and `transaction_id` with each order (needed for support and refunds).
- [ ] Customer-facing messages are ready in English and Arabic.

SUPPORT

For credentials, staging test accounts, IP whitelisting or any question, contact the Tasheel technical team. Please include your `merchantId`, the `cart_id` and the time of the request when reporting an issue. Reference documentation: `github.com/TasHeelTechnology/merchant-api-documentation`